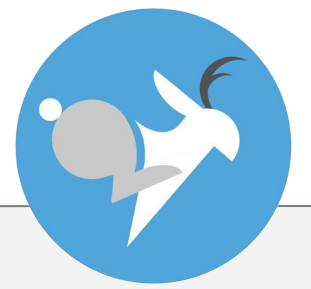
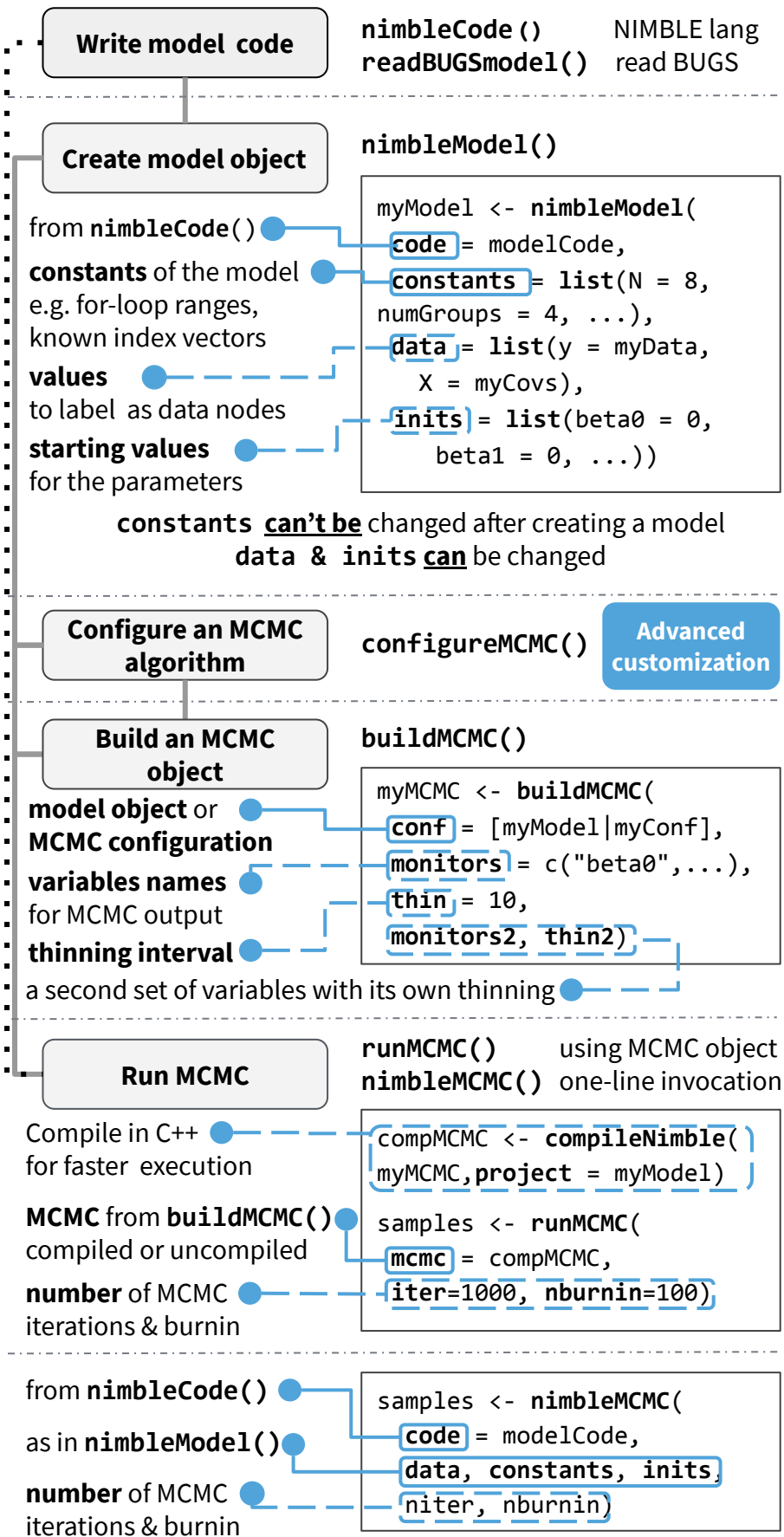
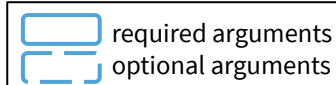


nimble models: : CHEAT SHEET



NIMBLE workflow



Writing model code

Split code over multiple lines to help people read it.

Use named arguments

for non-default parameterization
e.g. `beta0` and `beta1` follow
equivalent distributions
(default is precision, `tau`).

Link functions

can be declared on the
left-hand side.

Order of declaration does not matter

`alpha[iGroup]` can be
declared after being used in
other declarations.

```
modelCode <- nimbleCode(  
  beta0 ~ dnorm(0, sd = 1000)  
  beta1 ~ dnorm(0, 1E-6)  
  sdGroups ~ dunif(0, 100)  
  fixed_effects[1:N] <- beta0 + beta1 * X[1:N]  
  for(i in 1:N) {  
    log(eta[i]) <- fixed_effects[i] +  
      alpha[groupID[i]]  
    y[i] ~ dpois(eta[i])  
  }  
  for(iGroup in 1:numGroups) {  
    alpha[iGroup] ~ dnorm(0, sd = sdGroups)  
  }  
)
```

Vectorized declarations

create vector nodes. This means
`fixed_effects[1:N]` will be a
single node. One vector node vs.
multiple scalar nodes give different
model graphs, so use with care.

Provide explicit index ranges
or use empty brackets (`[]`) and
provide the **dimensions** argument
to `nimbleModel()`.

Nested indexing is a good way to
implement experimental groups or
factor levels. If groups are known
from the design, include them in
constants.

Using models

Models can be compiled.

`cModel <- compileNimble(myModel)`
In methods below, "model" can be
`cModel` or `myModel`.

Models can access and change variables.

```
model$beta0 <- 5  
model[["beta0"]] <- 5
```

Models can simulate or calculate log-probabilities.

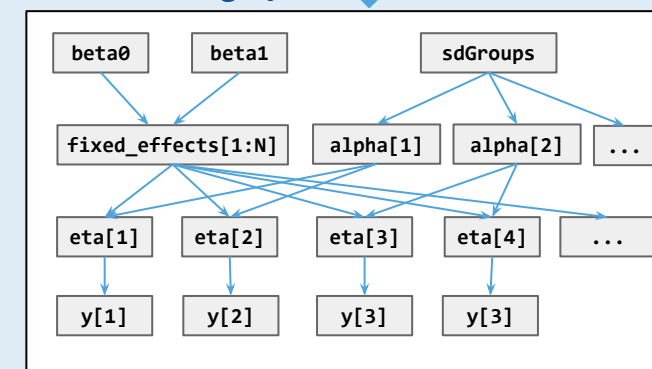
`model$calculate(nodes)`
returns sum of log probability densities.

`model$calculateDiff(nodes)`
returns difference in sum of log
probability densities between current and
previous node values.

`model$getLogProb(nodes)`
returns sum of most recently calculated
log probability densities.

`model$simulate(nodes,
 includeData = FALSE)`
simulates into stochastic nodes.
`includeData = FALSE` protects data.

Models are graphs



Uncompiled models can be debugged, updated, and copied.

Flag nodes as data and set inits

```
myModel$setData("y")  
myModel$setInits(inits)
```

Debug model errors

```
myModel$check()  
check for missing/invalid values.  
myModel$initializeInfo()  
which nodes are not fully initialized?
```

```
myModel$checkBasics()  
check for size/dimension mismatches and NA.
```

Make a copy

```
myModel$newModel(replicate = TRUE)
```

Models know properties of nodes.

```
model$getDimension(node)  
model$getDistribution(nodes)  
model$isDeterm(nodes)  
model$isStoch(nodes)  
model$isData(nodes)  
model$isDiscrete(nodes)  
model$isMultivariate(nodes)  
model$isBinary(nodes)  
model$isEndNode(nodes)  
model$isTruncated(nodes)
```

Models know about nodes, variables and relationships.

```
model$getNodeNames()  
returns node names  
e.g. "eta[1]", "eta[2]", ...
```

```
model$getVarNames()  
returns variable names  
e.g. "eta"
```

```
model$expandNodeNames(nodes)  
e.g. "y" is expanded to "y[1]", "y[2]", ...
```

```
model$getDependencies(nodes, ...)  
returns nodes that depend on input  
nodes.
```

nimble distributions and functions: : CHEAT SHEET



Declarations

STOCHASTIC

$x \sim \text{ddist}(\text{args})$

DETERMINISTIC

$z \leftarrow \text{fn}(\text{args})$

TRUNCATED STOCHASTIC

$x \sim T(\text{ddist}(\text{args}), \text{min}, \text{max})$

CENSORED STOCHASTIC

$\text{seg} \sim \text{dinterval}(t, c[1:n\text{Segments}])$

$t \sim \text{ddist}(\text{args})$

CONSTRAINT

$\text{one} \sim \text{dconstraint}(\text{condition})$

Deterministic Functions

SCALAR or COMPONENT-WISE

Logical: $|$, $\&$, $!$, $>$, $>=$, $<$, $<=$, $!=$,
 $==$, equals , step

Arithmetic: $+$, $-$, $*$, $/$, $^$, $\text{pow}(x, y)$
 $\%$, exp , log , sqrt , abs , cube

Trigonometric: sin , cos , tan , asin ,
 acos , atan , asinh , acosh , atanh

Links: logit , probit , cloglog
(links can also be used on left-hand side of a declaration)

Inverse links: $\text{ilogit}/\text{expit}$,
 $\text{iprobbit}/\text{phi}$, icloglog

Rounding: ceiling , floor , round ,
 trunc

Specials: $\text{lgamma}/\text{loggam}$, besselK ,
 loglp , lfactorial , logfact

Distributions: d , p , q , r forms of available
distributions can be used as deterministic
functions.

VECTOR and/or MATRIX

Returning scalar: inprod , logdet , sum ,
 mean , sd , prod , min , max

Returning vector: pmin , pmax ,
 $\text{eigen}(x)\$values$, $\text{svd}(x)\$d$

Returning matrix: inverse , chol , $\%*\%$,
 t , solve , forwardsolve ,
 backsolve , $\text{eigen}(x)\$vectors$,
 $\text{svd}(x)\$u$, $\text{svd}(x)\$v$

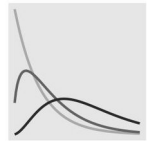
Univariate Distributions

Continuous



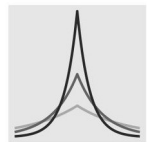
BETA

$y \sim \text{dbeta}([\text{shape1}, \text{shape2} \mid$
 $\text{mean}, \text{sd}])$
 $\text{shape1} = \text{mean}^2 * (1 - \text{mean}) / \text{sd}^2 - \text{mean}$
 $\text{shape2} = \text{mean} * (1 - \text{mean})^2 / \text{sd}^2 + \text{mean} - 1$



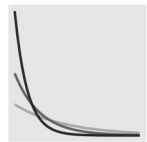
CHI-SQUARE

$y \sim \text{dchisq}(\text{df})$



DOUBLE EXPONENTIAL (LAPLACE)

$y \sim \text{ddexp}(\text{location},$
 $[\text{scale} \mid \text{rate} \mid \text{var}])$
 $\text{scale} = 1/\text{rate}$
 $\text{scale} = \text{sqrt}(\text{var}/2)$



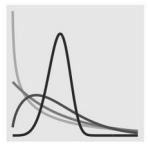
EXPONENTIAL

$y \sim \text{dexp}([\text{rate} \mid \text{scale}])$
 $\text{rate} = 1/\text{scale}$



FLAT (improper)

$y \sim \text{dflat}()$



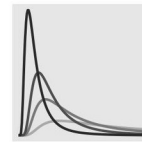
GAMMA

$y \sim \text{dgamma}([\text{shape}, [\text{rate} \mid \text{scale}]]$
 $[\text{mean}, \text{sd}])$
 $\text{scale} = 1/\text{rate}$
 $\text{shape} = \text{mean}^2 / \text{sd}^2$
 $\text{scale} = \text{sd}^2 / \text{mean}$



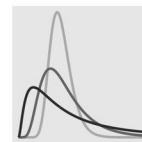
HALF FLAT (improper)

$y \sim \text{dhalfflat}()$



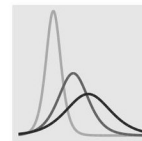
INVERSE GAMMA

$y \sim \text{dinvgamma}(\text{shape},$
 $[\text{rate} \mid \text{scale}])$
 $\text{rate} = 1/\text{scale}$



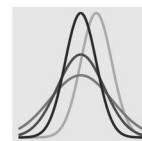
LOGISTIC

$y \sim \text{dlogis}(\text{location},$
 $[\text{rate} \mid \text{scale}])$
 $\text{scale} = 1/\text{rate}$



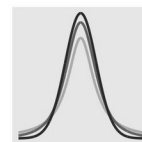
LOG-NORMAL

$y \sim \text{dlnorm}(\text{meanlog},$
 $[\text{taulog} \mid \text{sdlog} \mid \text{varlog}])$
 $\text{sdlog} = 1/\text{sqrt}(\text{taulog})$
 $\text{sdlog} = \text{sqrt}(\text{varlog})$



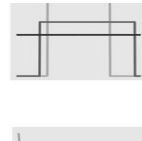
NORMAL

$y \sim \text{dnorm}(\text{mean},$
 $[\text{tau} \mid \text{sd} \mid \text{var}])$
 $\text{sd} = 1/\text{sqrt}(\text{tau})$
 $\text{sd} = \text{sqrt}(\text{var})$



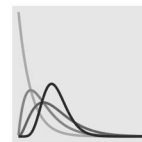
STUDENT T

$y \sim \text{dt}(\mu, [\text{tau} \mid \text{sigma} \mid \text{sigma2}], \text{df})$
 $\text{sigma} = 1/\text{sqrt}(\text{tau})$
 $\text{sigma} = \text{sqrt}(\text{sigma2})$



UNIFORM

$y \sim \text{dunif}(\text{min}, \text{max})$



WEIBULL

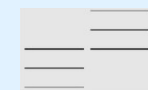
$y \sim \text{dweib}(\text{shape},$
 $[\text{lambda} \mid \text{scale} \mid \text{rate}])$
 $\text{scale} = \text{lambda}^{(-1/\text{shape})}$
 $\text{scale} = 1/\text{rate}$

DISTRIBUTION NAME

$y \sim \text{ddist}([\text{default} \mid \text{alternative}])$
 $\text{canonical} = \text{fn}(\text{provided})$

Lifted nodes are inserted when non-canonical
parameters are used. Default parameters are not
necessarily canonical.

Discrete



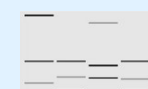
BERNOULLI

$y \sim \text{dbern}(\text{prob})$



BINOMIAL

$y \sim \text{dbinom}(\text{prob}, \text{size})$



CATEGORICAL

$y \sim \text{dcat}(\text{prob})$



NEGATIVE BINOMIAL

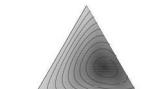
$y \sim \text{dnegbin}(\text{prob}, \text{size})$



POISSON

$y \sim \text{dpois}(\text{lambda})$

Multivariate distributions



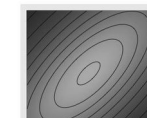
DIRICHLET

$y[] \sim \text{ddirch}(\text{alpha}[])$



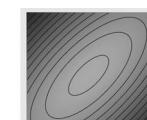
MULTINOMIAL

$y[] \sim \text{dmulti}(\text{prob}[], \text{size})$



MULTIVARIATE NORMAL

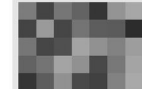
$y[] \sim \text{dmnorm}(\text{mean}[],$
 $[\text{prec}[], \text{cov}[] \mid \text{cholesky}[],])$,
 prec_param



MULTIVARIATE STUDENT T

$y[] \sim \text{dmvt}(\mu[],$
 $[\text{prec}[], \text{scale}[] \mid \text{cholesky}[],])$,
 df , prec_param

$\text{cholesky} = \text{chol}(\text{prec})$: $\text{prec_param}=1$
 $\text{cholesky} = \text{chol}(\text{cov})$: $\text{prec_param}=0$ for dmnorm
 $\text{cholesky} = \text{chol}(\text{scale})$: $\text{prec_param}=0$ for dmvt
 cholesky is $\text{chol}(\text{prec})$ when $\text{prec_param}=1$,
 $\text{chol}(\text{cov}) \mid \text{chol}(\text{scale})$ when $\text{prec_param}=0$



WISHART

$y[,] \sim \text{dwish}(\text{R}[,] \mid \text{S}[,] \mid \text{cholesky}[,]),$
 df , scale_param

INVERSE WISHART

$y[,] \sim \text{dinvwish}([\text{S}[,] \mid \text{R}[,] \mid \text{cholesky}[],])$,
 df , scale_param

$\text{cholesky} = \text{chol}(\text{R})$: $\text{scale_param}=0$
 $\text{cholesky} = \text{chol}(\text{S})$: $\text{scale_param}=1$
 cholesky is $\text{chol}(\text{S})$ when $\text{scale_param}=1$,
 $\text{chol}(\text{R})$ when $\text{scale_param}=0$

Distributions for spatial models



CONDITIONAL AUTOREGRESSIVE intrinsic (improper)

$y[] \sim \text{dcar_normal}(\text{adj}[],$
 $\text{weights}[], \text{num}[],$
 tau , c , zero_mean)

See Ch 9 of
User Manual

proper

$y[] \sim \text{dcar_proper}(\mu[], \text{C}[],$
 $\text{adj}[], \text{num}[], \text{M}[],$
 tau , gamma)

Bayesian nonparametric distributions

See Ch 10 of
User Manual



CHINESE RESTAURANT PROCESS

$y[] \sim \text{dCRP}(\text{conc}, \text{size})$
 conc = concentration parameter



STICK BREAKING PROCESS

$y[] \sim \text{stick_breaking}(z[])$
 z = vector of breaking points

Write you own!

See Ch 12 of
User Manual

NIMBLE allows you to write **new distributions**
and **functions** using `nimbleFunction()`.